

Self-Healing Data Pipeline

Deployment and Testing Guide

For: Shai Leib, CA DHCS ETS Data Architect

Author: Christian Abdel-Nour, Snowflake Solutions Delivery

Date: July 28, 2026

Repo: github.com/sfc-gh-cabdelnour/selfhealing-pipeline

1. What This Is

A Snowflake-native self-healing data pipeline that:

1. **Detects** when upstream BRONZE table schemas change (columns added, dropped, or type-changed)
2. **Halts** the pipeline to prevent broken data from propagating
3. **Diagnoses** the root cause using Cortex AI (claude-4-sonnet)
4. **Regenerates** downstream model SQL using Cortex LLM (llama3.1-70b)
5. **Validates** the fix by executing generated code against a zero-copy clone of production
6. **Opens a GitHub PR** with diagnosis + code + test results for human approval

The entire system runs inside Snowflake — no external orchestrator, no Lambda, no Airflow.

Design Rationale — Why Snowflake-Native: Every component (Tasks, stored procedures, Cortex AI, CLONE, External Access Integration) is a first-party Snowflake feature. This means: no infrastructure to manage, no credential rotation for external CI tools, no network egress except the GitHub API call, and the same RBAC governs the pipeline as governs the data.

2. Prerequisites

Required before deployment:

- **Snowflake account** with Enterprise edition or higher (for Cortex AI functions)
- **Snowflake CLI** (`snow`) installed and configured with a connection to your account
- **ACCOUNTADMIN** role access (or a role with CREATE DATABASE, CREATE ROLE, CREATE INTEGRATION privileges)
- **Cortex AI enabled** in your region (verify: `SELECT SNOWFLAKE.CORTEX.COMPLETE('llama3.1-70b', 'test');`)
- **[Optional] GitHub PAT** for PR automation (the pipeline works without it — PR step is skipped)
- **Git** to clone the repository

Design Rationale — ACCOUNTADMIN requirement: The pipeline creates databases, roles, and External Access Integrations — operations that require elevated privileges. In production, you'd create a dedicated admin role with only the required CREATE privileges. The foundation script creates SELFHEALING_ADMIN and SELFHEALING_EXECUTOR roles for least-privilege operation after initial setup.

3. Architecture

Object Inventory (what gets created)

Schema	Object	Type	Purpose
--------	--------	------	---------

CONFIG	SCHEMA_REGISTRY	Table	Known-good schema baseline (what columns SHOULD exist)
CONFIG	SCHEMA_CHANGE_EVENTS	Table	Detected drift events with lifecycle status
CONFIG	PIPELINE_HEALTH_AUDIT	Table	Data quality findings + AI diagnosis
CONFIG	PIPELINE_RUN_LOG	Table	Execution observability (when, how long, what happened)
CONFIG	ARTIFACT_REGISTRY	Table	dbt model lineage graph (what depends on what)
CONFIG	GENERATED_CODE	Table	LLM-regenerated SQL + validation results
CONFIG	VW_ARTIFACT_LINEAGE	View	Recursive CTE: given a table, find all downstream models
CONFIG	SP_SCHEMA_DRIFT_DETECTOR	Procedure	Compares INFORMATION_SCHEMA vs registry
CONFIG	SP_HALT_GATE	Procedure	Blocks pipeline on unresolved issues
CONFIG	FN_DIAGNOSE_FAILURE	Function	Cortex AI root cause analysis
CONFIG	SP_GENERATE_ARTIFACT_CODE	Procedure (Python)	LLM code regeneration
CONFIG	SP_VALIDATE_GENERATED_CODE	Procedure (Python)	Clone + execute + report
CONFIG	TASK_SELF_HEALING_PIPELINE	Task	Scheduled execution (15 min, suspended by default)
BRONZE	CUSTOMERS, ORDERS, ORDER_ITEMS	Tables	Demo source data
SILVER	ORDERS, ORDER_ITEMS	Views	Cleansed/normalized layer
GOLD	ORDERS_DAILY, CATEGORY_SUMMARY	Tables	Aggregate/reporting layer

Design Rationale — Separation of CONFIG schema: All pipeline infrastructure lives in CONFIG, completely separate from the data layers (BRONZE/SILVER/GOLD). This means: (1) the pipeline can be dropped without affecting data, (2) grants can be scoped differently (operators get CONFIG access, analysts get GOLD read-only), (3) the pattern is portable — drop CONFIG into any database.

4. Deployment Steps

Step 1: Clone the repository

```
git clone https://github.com/sfc-gh-cabdelnour/selfhealing-pipeline.git
cd selfhealing-pipeline
```

Step 2: Set your Snowflake connection

```
# If using Snowflake CLI:
export SNOWFLAKE_CONNECTION=your-connection-name

# Verify connectivity:
snow sql -c $SNOWFLAKE_CONNECTION -q "SELECT CURRENT_ACCOUNT(), CURRENT_ROLE();"
```

Step 3: Deploy foundation (roles, database, schemas, tables)

```
snow sql -c $SNOWFLAKE_CONNECTION -f config/01_foundation.sql
```

This creates:

- Roles: `SELFHEALING_ADMIN` , `SELFHEALING_EXECUTOR`
- Database: `SELFHEALING_PROD` with 4 schemas
- Warehouse: `SELFHEALING_WH` (XSMALL, auto-suspend 60s)
- All 7 CONFIG tables with constraints and retention policies

Design Rationale — CREATE IF NOT EXISTS everywhere: The foundation script is idempotent. Running it twice does nothing harmful. This is critical for operational safety — if someone accidentally re-runs deployment, it won't drop existing data. Compare to `CREATE OR REPLACE` which would destroy everything.

Design Rationale — DATA_RETENTION_TIME_IN_DAYS: Event and audit tables have 30-90 day retention. This enables Time Travel for recovery and ensures old resolved events are garbage-collected automatically. Without this, tables grow indefinitely.

Step 4: Deploy detection procedures

```
snow sql -c $SNOWFLAKE_CONNECTION -f config/02_detection.sql
```

Design Rationale — Detection via INFORMATION_SCHEMA diff: We compare the live schema (`INFORMATION_SCHEMA.COLUMNS`) against a stored baseline (`SCHEMA_REGISTRY`). This is pure SQL — no external catalog, no Horizon integration required. It catches: new columns, dropped columns, and data type changes. The detector is idempotent: running it 10 times with no schema change produces zero events.

Step 5: Deploy diagnosis + halt gate

```
snow sql -c $SNOWFLAKE_CONNECTION -f config/03_diagnosis.sql
```

Design Rationale — TRY_PARSE_JSON fallback: The AI diagnosis function uses `TRY_PARSE_JSON` instead of `PARSE_JSON` . If the LLM returns malformed JSON (it happens ~5% of the time), the function returns a safe fallback object instead of crashing the entire pipeline. Production-grade error handling.

Design Rationale — Halt gate as separate procedure: The gate is a standalone check that any pipeline step can call. Want to add it to your existing ingestion pipeline? Just add `CALL SP_HALT_GATE();` at the top of your ingestion stored procedure and check for 'HALTED' in the return value. The gate doesn't know or care about the rest of the self-healing logic.

Step 6: Deploy lineage + code generation

```
snow sql -c $SNOWFLAKE_CONNECTION -f config/04_lineage_codegen.sql
```

Design Rationale — VW_ARTIFACT_LINEAGE as a view: The recursive CTE is defined once as a view and queried by all downstream procedures. This eliminates code duplication (the original version had the same CTE copy-pasted in 2 places). The view is always current — any change to ARTIFACT_REGISTRY is immediately reflected.

Design Rationale — Python for code generation: We use a Python stored procedure for the codegen loop because Snowflake SQL Scripting has limitations with cursor variable binding inside FOR loops (cursor row fields like `rec.COLUMN_NAME` are inconsistently accessible). Python with Snowpark gives us reliable iteration, proper error handling, and cleaner Cortex integration. The tradeoff: Python procs have ~2s cold-start overhead vs SQL procs. Acceptable for a 15-minute scheduled task.

Design Rationale — Idempotent code generation: The procedure checks `if session.table(...).filter(...).count() > 0: continue` before generating. If the task runs twice for the same event, it skips already-generated artifacts. This prevents duplicate rows in GENERATED_CODE and wasted Cortex credits.

Step 7: Deploy GitHub integration (optional)

```
# Only if you have a GitHub PAT and want PR automation:
# First, create the secret (run in Snowsight):
#   CREATE OR REPLACE SECRET SELFHEALING_PROD.CONFIG.GITHUB_PAT_SECRET
#   TYPE = GENERIC_STRING SECRET_STRING = 'ghp_yourPAThere';

snow sql -c $SNOWFLAKE_CONNECTION -f config/05_github_eai.sql
```

Note: The pipeline works without the GitHub integration. If you skip this step, the pipeline still detects, diagnoses, generates code, and validates — you just won't get automated PRs. Review generated code directly in the GENERATED_CODE table.

Design Rationale — External Access Integration (EAI): EAI is the only way for Snowflake to make outbound HTTP calls (to GitHub's API). It requires an explicit network rule (only `api.github.com:443` is allowed) and an authentication secret. This is the most locked-down approach — no broad internet access, no wildcard rules, no long-lived tokens stored in code.

Step 8: Deploy validation

```
snow sql -c $SNOWFLAKE_CONNECTION -f config/06_validation.sql
```

Design Rationale — Zero-copy clone for testing: `CREATE DATABASE ... CLONE` creates an instant, full-fidelity copy of production with zero storage cost (copy-on-write). We execute generated SQL against this clone to prove it works against real data structures. The clone is dropped immediately after validation to prevent storage accumulation. Total cost: ~0.001 credits per validation cycle.

Step 9: Deploy orchestrator + task

```
snow sql -c $SNOWFLAKE_CONNECTION -f config/07_pipeline_tasks.sql
```

Design Rationale — Task suspended by default: The scheduled task starts in SUSPENDED state. You must explicitly `ALTER TASK ... RESUME` to activate automated execution. This prevents unintended pipeline runs during deployment or testing. Resume only when you're confident the system is working correctly.

Step 10: Seed demo data

```
snow sql -c $SNOWFLAKE_CONNECTION -f config/08_seed_demo.sql
```

This populates BRONZE tables, creates SILVER/GOLD models, seeds the schema registry baseline, and registers the artifact lineage graph.

5. Testing the Deployment

Test 1: Verify all objects exist

```
-- Should return 7 CONFIG tables + 1 view
SELECT TABLE_NAME, TABLE_TYPE FROM SELFHEALING_PROD.INFORMATION_SCHEMA.TABLES
WHERE TABLE_SCHEMA = 'CONFIG' ORDER BY TABLE_TYPE, TABLE_NAME;

-- Should return 5 procedures + 1 function
SHOW PROCEDURES IN SCHEMA SELFHEALING_PROD.CONFIG;
SHOW USER FUNCTIONS IN SCHEMA SELFHEALING_PROD.CONFIG;
```

Test 2: Simulate schema drift end-to-end

```
USE SCHEMA SELFHEALING_PROD.CONFIG;

-- Simulate column addition
ALTER TABLE SELFHEALING_PROD.BRONZE.ORDERS ADD COLUMN LOYALTY_POINTS NUMBER;

-- Detect
CALL SP_SCHEMA_DRIFT_DETECTOR();
-- Expected: NEW=1

-- Halt
CALL SP_HALT_GATE();
-- Expected: HALTED - 0 critical, 1 pending

-- Get event ID
SET eid = (SELECT EVENT_ID FROM SCHEMA_CHANGE_EVENTS WHERE PIPELINE_STATUS='PENDING' LIMIT 1);

-- Generate code (takes 30-60 seconds for Cortex)
CALL SP_GENERATE_ARTIFACT_CODE($eid);
-- Expected: Generated 1 artifacts

-- Validate
CALL SP_VALIDATE_GENERATED_CODE($eid);
-- Expected: PASS=0 FAIL=1 (GOLD aggregate model can't auto-resolve)

-- Check results
SELECT ARTIFACT_NAME, TEST_STATUS, LEFT(TEST_ERROR, 100) FROM GENERATED_CODE WHERE EVENT_ID = $eid;
```

Test 3: Verify idempotency

```

-- Run detector again - should find 0 new events (already tracked)
CALL SP_SCHEMA_DRIFT_DETECTOR();
-- Expected: NEW=0

-- Run codegen again - should skip (already generated)
CALL SP_GENERATE_ARTIFACT_CODE($eid);
-- Expected: Generated 0 artifacts

```

Test 4: Clean up and re-test (full teardown/rebuild)

```

-- Teardown
DROP DATABASE IF EXISTS SELFHEALING_PROD;
DROP DATABASE IF EXISTS SELFHEALING_DEV;
DROP WAREHOUSE IF EXISTS SELFHEALING_WH;
DROP ROLE IF EXISTS SELFHEALING_EXECUTOR;
DROP ROLE IF EXISTS SELFHEALING_ADMIN;

-- Rebuild: re-run Steps 3-10 above
-- Then re-run Test 2
-- Results should be identical (deterministic)

```

6. Adapting for Your Environment

To use with your existing dbt project:

1. **Populate ARTIFACT_REGISTRY** with your real model graph. Each row = one model + what it depends on. You can generate this from `dbt ls --output json`.
2. **Seed GENERATED_CODE baseline** with the current SQL for each model. This gives the LLM context for regeneration.
3. **Point SCHEMA_REGISTRY** at your actual BRONZE schema: `INSERT INTO SCHEMA_REGISTRY SELECT ... FROM YOUR_DB.INFORMATION_SCHEMA.COLUMNS WHERE TABLE_SCHEMA = 'BRONZE'`.
4. **Update the VW_ARTIFACT_LINEAGE view** if your lineage has more than 2 hops (adjust the DEPTH limit in the recursive CTE).

Design Rationale — Decoupled from dbt: The pipeline doesn't require dbt to run. It operates on plain SQL and a metadata table (ARTIFACT_REGISTRY) that describes the lineage graph. If you use dbt, populate the registry from the manifest. If you use stored procedures, populate it manually. If you use Dynamic Tables, populate it from SHOW DYNAMIC TABLES dependencies. The pattern is framework-agnostic.

To integrate the halt gate into existing pipelines:

```

-- Add this to the TOP of any ingestion stored procedure:
DECLARE v_gate VARCHAR;
BEGIN
    CALL SELFHEALING_PROD.CONFIG.SP_HALT_GATE() INTO v_gate;
    IF (LEFT(v_gate, 6) = 'HALTED') THEN
        -- Log and exit without processing
        RETURN 'Pipeline blocked: ' || v_gate;
    END IF;
    -- ... rest of your ingestion logic ...
END;

```

7. Design Decision Summary

Decision	Choice	Rationale
Language for detection	SQL	Pure set operations on INFORMATION_SCHEMA — no iteration needed, fastest cold-start
Language for codegen/validation	Python (Snowpark)	Reliable cursor iteration, proper error handling, cleaner Cortex integration
LLM for diagnosis	claude-4-sonnet	Best reasoning capability for root cause analysis with structured output
LLM for code generation	llama3.1-70b	Good SQL generation, lower cost, sufficient for mechanical rewrites
Lineage storage	Table + recursive view	SQL-native, no external manifest parsing, always current
Validation approach	Zero-copy clone + execute	Tests against real data structures at zero storage cost
Human gate	PR-first (GitHub EAI)	Every change is auditable, reviewable, and reversible via standard git workflow
Scheduling	Snowflake Task (15 min)	Native, no external scheduler, auto-suspend warehouse between runs
Idempotency	Skip-if-exists on all writes	Safe for re-execution after transient failures without duplicating work
Error handling	TRY_PARSE_JSON + Python try/except	Graceful degradation — pipeline logs errors but doesn't crash on LLM issues

8. Troubleshooting

Symptom	Cause	Fix
SP_GENERATE_ARTIFACT_CODE returns "No downstream for TABLE"	ARTIFACT_REGISTRY missing entry for that table	Insert the lineage row: <code>INSERT INTO ARTIFACT_REGISTRY VALUES ('MODEL', 'SCHEMA', 'path', 'TABLE', 'TABLE', CURRENT_TIMESTAMP())</code>
FN_DIAGNOSE_FAILURE returns fallback JSON (confidence: 0.0)	Cortex returned malformed response	Check Cortex availability: <code>SELECT SNOWFLAKE.CORTEX.COMPLETE('claude-4-sonnet', 'test');</code> — may be region-limited or rate-limited
SP_VALIDATE_GENERATED_CODE returns "Clone failed"	Insufficient privileges to clone database	Ensure running as ACCOUNTADMIN or role with CREATE DATABASE privilege
Detector finds 0 changes after ALTER TABLE	Event already exists for that column (idempotency guard)	Check SCHEMA_CHANGE_EVENTS for existing PENDING entry. Resolve or delete it first.
GitHub PR fails with 401	PAT expired or insufficient scope	Regenerate PAT with <code>repo</code> scope. Update secret: <code>ALTER SECRET ... SET SECRET_STRING = '...'</code>

9. Credits and References

- **Pattern origin:** Tim Buchhorn (Snowflake SE, APJ) — [selfhealing-demo](#) (detection + lineage + dbt integration)

- **Enhancements:** Christian Abdel-Nour (Snowflake SD) — halt gate, AI diagnosis with confidence scoring, health checks beyond schema drift, production hardening (idempotency, TRY_PARSE_JSON, custom roles, retention policies)
- **Cortex AI best practices:** SKE Confluence — "Start with capable models for baseline, optimize for cost"